GitHub 저장소 만들기 초보자 실습 가이드

명령어 세 줄과 프롬프트 한 줄로 내 저장소를 만듭니다

설치

↓

로그인

↓

저장소 생성

↓

매일 갱신

Claude Code로 따라 하는 GitHub 첫걸음

네 단계로 처음부터 끝까지 따라 합니다

01	기본 개념	**Git과 GitHub** 로컬 저장소 · 원격 저장소 · 클코의 역할
02	사전 준비	**최초 1회** Git · GitHub CLI · Claude Code 설치 · 로그인
03	저장소 만들기	**실습** 폴더 준비 · 프롬프트 · .gitignore · 확인
04	갱신과 문제 해결	**매일 쓰는 법** 커밋·push · 되돌리기 · 자주 막히는 지점
05	실전 예시	**과정 저장소** clone · .gitignore · CLAUDE.md · GitHub Pages

준비 30분, 이후에는 프롬프트 한 줄

01

기본 개념

명령을 치기 전에 Git과 GitHub가 무엇인지 먼저 구분한다

Git은 기록 도구이고 GitHub는 인터넷 보관소입니다

둘은 이름이 비슷하지만 서로 다른 것이다 — 하나는 프로그램, 하나는 웹사이트

GIT · 내 컴퓨터

버전 기록 프로그램

- 파일 변경 이력 저장
- 언제든 이전으로 복구
- 인터넷 없이 동작



GITHUB · 인터넷

저장소 보관 웹사이트

- 기록을 온라인에 백업
- 다른 PC·동료와 공유
- 웹페이지 배포 출발점

저장소(repository) = 프로젝트 폴더 + 변경 이력 전체

파일은 네 칸을 거쳐 GitHub에 도착합니다

add(고르기) → commit(기록) → push(올리기) — 용어는 이 세 개면 충분하다



클코에게 한 줄로 지시하면 이 세 명령을 순서대로 대신 실행합니다

클코는 명령을 실행하기 전에 반드시 허락을 구합니다

Claude Code(클코) = 터미널에서 한국어 지시를 받아 명령을 대신 실행하는 AI



※ 무슨 명령인지 모르겠으면 승인하기 전에 "이 명령이 뭘 하는지 설명해줘"라고 물어본다

02

사전 준비

계정 1개와 프로그램 3개를 설치하고 로그인한다 — 컴퓨터마다 최초 1회

준비물은 계정 하나와 프로그램 세 개입니다

Claude Code는 Claude 유료 구독(Pro 이상) 계정으로 로그인한다

계정	프로그램 1	프로그램 2	프로그램 3
GitHub 계정	**Git**	**GitHub CLI**	**Claude Code**
github.com 가입	버전 기록 도구	gh 명령	claude 명령

※ 터미널 여는 법 — Mac: Spotlight(⌘+Space)에서 "터미널" / Windows: 시작 메뉴에서 "PowerShell"

GitHub CLI(gh)가 있어야 클코가 내 계정에 저장소를 만들 수 있습니다

Mac은 터미널에 세 줄을 붙여넣으면 설치가 끝납니다

한 줄씩 붙여넣고 Enter — 비밀번호를 물으면 Mac 로그인 비밀번호(화면에 안 보임)

macOS · 터미널

`xcode-select --install`	① Git (개발 도구 설치 창 → 설치)
`brew install gh`	② GitHub CLI
`curl -fsSL https://claude.ai/install.sh | bash`	③ Claude Code

※ brew: command not found가 나오면 brew.sh 첫 화면의 설치 명령을 먼저 붙여넣고 ②를 다시 한다

설치 직후에는 터미널을 닫고 새로 열어야 명령이 인식됩니다

Windows는 PowerShell에 세 줄을 붙여넣으면 설치가 끝납니다

시작 메뉴에서 PowerShell 실행 — 화면 앞에 PS가 보이면 맞게 연 것이다

Windows · PowerShell

`winget install --id Git.Git -e`	① Git
`winget install --id GitHub.cli`	② GitHub CLI
`irm https://claude.ai/install.ps1 | iex`	③ Claude Code

※ 설치 중 "동의하시겠습니까?"가 나오면 Y 입력 후 Enter

설치 직후에는 PowerShell을 닫고 새로 열어야 명령이 인식됩니다

gh auth login 한 번이면 내 GitHub 계정과 연결됩니다

터미널에 gh auth login 입력 → 네 개 질문에 아래처럼 답한다



화살표 키(↑ ↓)로 고르고 Enter. 마지막에 화면의 일회용 코드를 복사해 둔다

끝나면 Logged in as 내아이디 가 보이면 성공

Git에는 처음 한 번 내 이름과 이메일을 알려 줍니다

커밋(기록)마다 작성자로 남는 정보 — GitHub 가입 이메일과 같게 쓴다

터미널에 한 줄씩 입력 — 따옴표 안만 내 정보로 바꾼다

`git config --global user.name "Hong Gildong"`	이름 (영문 권장)
`git config --global user.email "me@example.com"`	가입 이메일
`git config --global init.defaultBranch main`	기본 가지 = main

명령이 어렵다면 — 클코 실행 후 이렇게 시킨다

클코에 그대로 붙여넣기

git 사용자 이름을 Hong Gildong, 이메일을 me@example.com으로 전역 설정해줘.

한 번 설정하면 이 컴퓨터의 모든 저장소에 적용됩니다

네 줄로 준비가 끝났는지 확인합니다

버전 숫자가 나오면 설치 성공 — "command not found"가 나오면 그 프로그램을 다시 설치한다

입력할 명령 →	성공하면 보이는 화면
`git --version`	git version 2.x.x
`gh --version`	gh version 2.x.x
`gh auth status`	Logged in to github.com account 내아이디
`claude --version`	버전 번호 (예: 2.x.x)

네 줄이 모두 통과해야 3부 실습으로 넘어갑니다

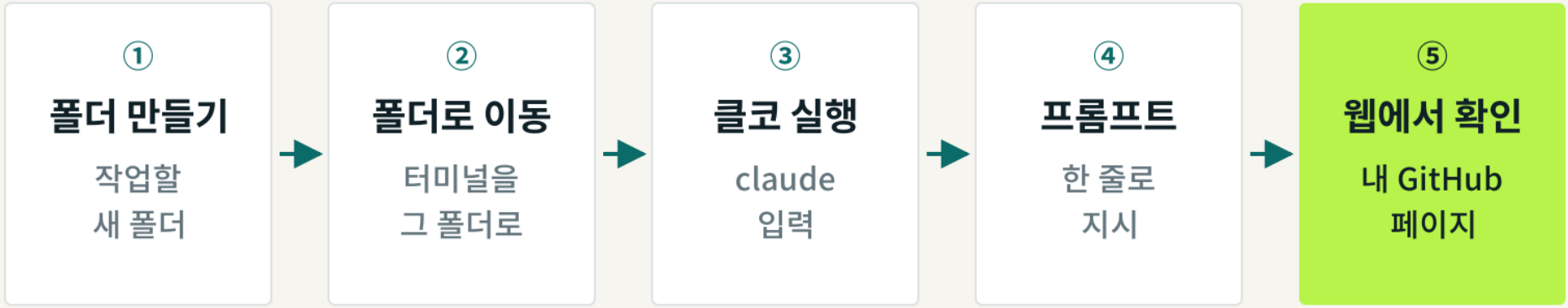
03

저장소 만들기

폴더를 만들고 클코를 실행한 뒤 프롬프트 한 줄로 GitHub에 올린다

저장소는 다섯 단계로 만들어집니다

예시 폴더 이름: my-notes (영문·숫자·하이픈만 쓴다)



명령어는 ①②③만 직접 칩니다. 나머지는 클코가 합니다

직접 치는 명령은 세 줄뿐입니다

① ② ③ 단계 — Mac·Windows 공통 (홈 폴더 아래에 만든다)

터미널에 한 줄씩 입력

`mkdir my-notes`	① 새 폴더 만들기
`cd my-notes`	② 그 폴더로 이동
`claude`	③ 클코 실행 (첫 실행 시 로그인)

첫 실행 질문 ①

폴더 신뢰 → Yes

첫 실행 질문 ②

Claude 계정 로그인

준비 완료

> 입력창 등장

폴더 안에서 claude를 실행해야 그 폴더가 저장소가 됩니다

프롬프트 한 번으로 저장소 생성과 업로드가 끝납니다

④ 단계 — 클코 입력창(>)에 붙여넣고 Enter, 허락을 물으면 Yes

프롬프트 ① 저장소 만들기

클코에 그대로 붙여넣기

이 폴더를 git 저장소로 초기화하고 GitHub에 비공개(private) 저장소
"my-notes"로 만들어서 push해줘.
README.md에는 이 저장소의 목적을 세 줄로 쓰고,
.gitignore에는 .env, .DS_Store, __pycache__/, data/ 를 넣어줘.
끝나면 저장소 주소를 알려줘.

※ 공개로 만들려면 "비공개(private)"를 "공개(public)"로 바꾼다. 나중에 웹 설정에서도 바꿀 수 있다

명령어를 몰라도 원하는 결과를 한국어로 쓰면 됩니다

클코는 내부적으로 다섯 개 명령을 실행합니다

외울 필요는 없다 — 허락 요청 화면에 이런 명령이 보이면 정상이다

명령 → 뜻	
`git init`	폴더를 저장소로 만든다
`git add .`	모든 파일을 올릴 목록에 담는다
`git commit -m "Initial commit"`	첫 버전으로 기록한다
`gh repo create my-notes --private --source=.`	GitHub 저장소 생성·연결
`git push -u origin main`	기록을 GitHub로 올린다

add · commit · push — 1부에서 본 세 칸 그대로입니다

.gitignore에 적은 파일은 GitHub에 올라가지 않습니다

비밀번호가 든 .env와 대용량 데이터는 처음부터 빼 둔다

GitHub에 올리는 것	
README.md	설명서
notes/	강의노트
*.py	코드
*.md	문서

올리지 않는 것 (.gitignore)	
.env	비밀번호·API 키
data/	대용량 데이터
.DS_Store	맥 시스템 파일
__pycache__/	임시 파일

※ 한 번 올라간 비밀번호는 지워도 이력에 남는다 — 그때는 키를 새로 발급받는다

브라우저에서 내 저장소를 열어 결과를 확인합니다

⑤ 단계 — 클코 안에서 묻거나, 터미널 명령으로 바로 연다

프롬프트 ② 결과 확인

클코에 그대로 붙여넣기

방금 만든 저장소를 웹 브라우저로 열어줘.
그리고 GitHub에 올라간 파일 목록을 보여줘.

같은 일을 명령으로 할 때

`gh repo view --web`	저장소 웹페이지 열기
`git status`	아직 안 올린 변경이 있는지 확인

웹에 README가 보이면 첫 저장소 완성입니다

04

갱신과 문제 해결

만든 뒤에는 매일 같은 프롬프트로 고친 내용을 올린다

저장소는 수정하고 올리는 순환으로 씁니다

commit+push = 내 PC → GitHub, pull = GitHub → 다른 PC



매일 쓰는 프롬프트는 네 개면 충분합니다

저장소 폴더에서 claude를 실행한 뒤 입력한다

일상 프롬프트

필요한 줄만 골라 입력

① 변경사항을 알아보기 쉬운 메시지로 커밋하고 push해줘.
② 최근 커밋 5개를 무엇을 바꿨는지 한국어로 요약해줘.
③ 마지막 커밋 이전 상태로 notes.md만 되돌려줘.
④ GitHub에 있는 최신 내용을 가져와줘. (git pull)

※ 다른 PC에서 처음 받을 때: "내아이디/my-notes 저장소를 이 폴더에 clone해줘"

작업이 끝날 때마다 ①번 한 줄이 백업입니다

막히는 지점은 대부분 다섯 가지입니다

오류 메시지를 그대로 클코에 붙여넣고 "원인과 해결 방법 알려줘"라고 해도 된다

증상	원인	해결
command not found	설치 안 됨·창 안 새로 엶	재설치 후 터미널 새로
permission denied	gh 로그인 안 됨	gh auth login 다시
name already exists	같은 이름 저장소 있음	이름을 바꿔 다시 지시
rejected (fetch first)	웹에서 먼저 수정됨	"pull 하고 다시 push해줘"
file too large	100MB 넘는 파일	.gitignore에 추가

오류는 고장이 아니라 클코에게 줄 다음 프롬프트입니다

05

실전 예시

이 과정의 저장소 kma-auto-genai-3-1을 받아 보고 같은 구조로 내 저장소를 만든다

과정 저장소는 자료·데이터·사이트를 한곳에 둡니다

github.com/keerhee/kma-auto-genai-3-1 — 날짜 폴더마다 강의 순서대로 번호를 붙였다

keerhee/kma-auto-genai-3-1

강의 자료	실습 데이터	만드는 도구	안내·규칙	웹 사이트
`Day0/ ~ Day6/` `Robot_Special/`	`data/` `data/README.md`	`_build/` `x_skills/`	`README.md` `.gitignore`	`index.html` `.nojekyll`
날짜별 PDF 덱	실데이터·가상	빌드 스크립트	첫 화면·제외 목록	GitHub Pages

자료(PDF)·데이터·사이트를 한 저장소에 두고, 폴더 이름에 순서를 붙인다

수업 저장소는 clone 한 줄로 내 PC에 받습니다

clone = 저장소 전체를 처음 한 번 복사, pull = 그 뒤 바뀐 부분만 받기

터미널에 한 줄씩 입력

`gh repo clone keerhee/kma-auto-genai-3-1`	처음 한 번 내려받기
`cd kma-auto-genai-3-1`	받은 폴더로 이동
`git pull`	강사가 올린 새 자료 받기

프롬프트 ③ 받은 저장소 둘러보기

클코에 그대로 붙여넣기

README.md를 읽고 이 저장소의 폴더 구성과 각 폴더의 역할을 표로 설명해줘.

공개 저장소는 내 계정이 아니어도 받을 수 있습니다

실제 과정 저장소는 원본 PPT를 빼고 PDF만 올립니다

.gitignore 실례 — 한 줄에 한 패턴, *는 '아무 글자나'라는 뜻이다

keerhee/kma-auto-genai-3-1 의 .gitignore

`.DS_Store`	Mac이 폴더마다 만드는 시스템 파일
`~$*`	Office 문서를 열면 생기는 임시 파일
`*.pptx`	원본 덱은 빼고 PDF만 공개
`node_modules/`	설치 패키지 — 용량이 크다
`__pycache__/`	파이썬 임시 파일

무엇을 올리지 않을지 정하는 것도 저장소 설계입니다

CLAUDE.md에 규칙을 적어 두면 클코가 매번 지킵니다

폴더의 CLAUDE.md를 클코가 시작할 때마다 읽는다 — Day2 실습 폴더의 실례

규칙 1

원본 보존

input/은 수정 금지

규칙 2

작업 분리

working/ → output/

규칙 3

숫자에 기준

기간·출처를 붙인다

프롬프트 ④ 규칙 파일 만들기

클코에 그대로 붙여넣기

이 저장소용 CLAUDE.md를 만들어줘. input/ 파일은 수정하지 않고,
중간 결과는 working/, 최종 결과는 output/에 둔다는 규칙을 넣어줘.

같은 말을 프롬프트마다 반복하지 않아도 됩니다

GitHub Pages를 켜면 저장소가 웹사이트가 됩니다

index.html이 첫 화면이 되고, .nojekyll은 파일을 가공 없이 공개하라는 표시다



주소 형식

https://아이디.github.io/저장소이름/

`keerhee.github.io/kma-auto-genai-3-1`

※ 무료 계정은 public 저장소에서만 Pages를 쓸 수 있다

같은 구조의 내 과정 저장소를 프롬프트 하나로 만듭니다

kma-auto-genai-3-1을 본보기로 삼아 폴더·README·사이트까지 한 번에 지시한다

프롬프트 ⑤ 내 과정 저장소 + 사이트

클코에 그대로 붙여넣기

keerhee/kma-auto-genai-3-1 저장소처럼 내 과정 저장소를 만들어줘.
Day1~Day3 폴더, data/ 폴더, 폴더 표가 들어간 README.md,
*.pptx를 제외하는 .gitignore, 폴더 카드가 보이는 index.html을 넣고
public 저장소 "my-course"로 push한 뒤 GitHub Pages를 켜줘.
끝나면 저장소 주소와 사이트 주소를 둘 다 알려줘.

※ 웹에서 직접 켤 때: 저장소 Settings → Pages → Branch를 main, /(root)로 두고 Save

본보기 저장소 하나가 가장 좋은 프롬프트입니다

한 장으로 정리하면 명령 세 줄과 프롬프트 두 개입니다

준비가 끝난 컴퓨터에서 새 저장소를 만들 때마다 이 장만 보면 된다

터미널 (직접 입력)

`mkdir 폴더이름`	새 폴더
`cd 폴더이름`	폴더로 이동
`claude`	클코 실행

클코 (붙여넣기)

클코에 그대로 붙여넣기

처음: 이 폴더를 저장소로 만들고 GitHub에 private으로 push해줘.
매일: 변경사항을 알아보기 쉬운 메시지로 커밋하고 push해줘.

명령은 시작할 때만, 나머지는 한국어로 지시합니다

INSTALL ONCE.

PROMPT IN KOREAN.

PUSH EVERY DAY.

본보기: github.com/keerhee/kma-auto-genai-3-1